

第 12 讲：多 Agent 与并行开发：收益、边界与协调成本

Coding Agent Notes (June 2026)

本讲在系列中的位置：#11 已经把 product coding agent 拆成 surface、workspace、permission、context、evidence 和 review。

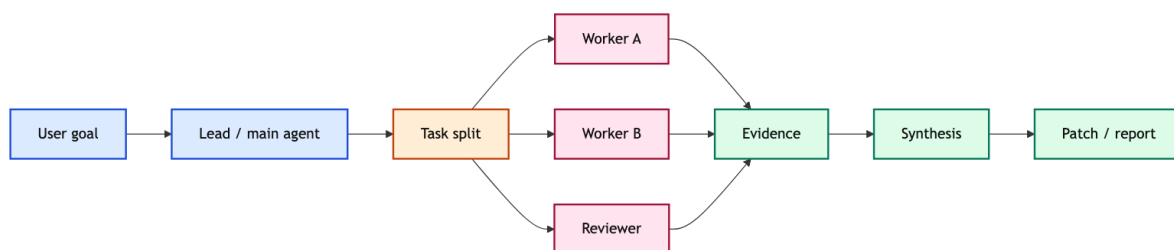
已知前提：读者已经知道 harness loop、worktree、review、权限和长任务状态。

本讲新增：multi-agent coding 的收益边界：并行证据、角色分工、任务切分、handoff、merge conflict、review 负担和协调成本。

读完后应该能回答：为什么多 agent 不是默认更强；怎样判断一个任务是否值得拆成多个 agent；怎样设计 worker 输出让最终 synthesis 可审查。

1 核心命题

multi-agent coding 最容易被误解成“多几个模型一起想，答案就更好”。这不是一个可靠的系统原则。更多 agent 会带来更多上下文复制、更多工具调用、更多权限提示、更多中间结论、更多合并冲突和更多 review 工作。它只有在一个条件下才划算：**被拆出去的工作足够独立，返回的结果足够可验证，主 agent 或人类 reviewer 能以较低成本合成这些结果。**



因此，多 agent 的基本结构不是群聊，而是一个 workflow：用户目标进入 lead 或 main agent；lead 把任务拆成边界明确的子任务；worker agent 独立收集证据或修改局部区域；reviewer 或 ranker agent 检查结果；最后 lead 把证据、diff、风险和建议合成为一个可交付对象。

MASAI、ChatDev、MetaGPT、AutoGen、Codex subagents 和 Claude Code agent teams 看起来差异很大，但都在回答同一个问题：复杂任务怎样拆，拆完怎样沟通，沟通完怎样合成，合成后怎样验收。真正重要的不是 agent 数量，而是**任务边界和输出契约**。

Insight. 多 agent 的价值不来自“角色扮演”本身，而来自把长上下文和复杂任务切成可独立执行、可独立验证、可独立 review 的工作单元。没有清楚的输出契约，多 agent 只会把一个混乱任务变成多个混乱任务。

2 多 Agent 的三种收益

第一种收益是 wall-clock parallelism。多个 agent 可以同时读不同文件、跑不同测试、检查不同风险、调查不同假设。这个收益只在子任务真的独立时成立。如果两个 worker 都要改同一个核心文件，等待时间可能减少，但合并和 review 时间会增加。

第二种收益是 independent evidence。单个 agent 容易找到一个 plausible explanation 然后停止探索；多个 agent 可以分别从安全、性能、测试覆盖、日志、依赖、用户路径等角度检查同一问题。Claude

Code agent teams 文档里的并行 PR review 例子正是这种模式：不同 teammate 用不同 lens 看同一个 diff，lead 再合成 findings。

第三种收益是 context isolation。一个 worker 可以只关注日志，一个 worker 只关注测试，一个 worker 只读数据库迁移。这样每个上下文窗口更干净，减少 main thread 的 context pollution。Codex subagents 文档也把 read-heavy exploration、tests、triage、summarization 作为起点，这说明产品系统更鼓励把“读和判断”并行化，而不是一开始就让多个 agent 同时写代码。

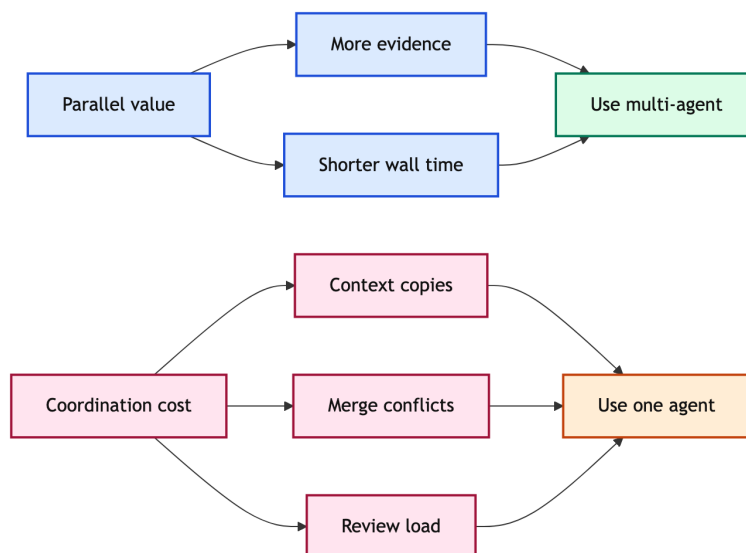
收益	适合的任务	关键条件
缩短等待时间	多测试、多文件扫描、多日志分析	子任务能并行完成
增加证据多样性	review、安全、性能、根因假设	每个 agent 有不同 lens
隔离上下文	大 repo 探索、文档总结、局部调查	worker 输出能压缩成摘要

3 协调成本模型

多 agent 是否值得用，可以用一个简单判断式：

$$\text{Net Gain} = \text{Parallel Value} - \text{Context Cost} - \text{Communication Cost} - \text{Merge Cost} - \text{Review Cost} - \text{Permission Cost}.$$

这个式子不是为了精确计算，而是为了阻止一个常见误区：只看并行收益，不看协调成本。Context cost 来自每个 agent 都要加载项目规则、文件、历史和工具状态。Communication cost 来自 lead 要写清楚任务、worker 要回报结果、系统要传递消息。Merge cost 来自多个 diff 可能冲突。Review cost 来自人类或 lead 要判断更多中间产物。Permission cost 来自多个 agent 同时请求命令、文件写入或网络访问。

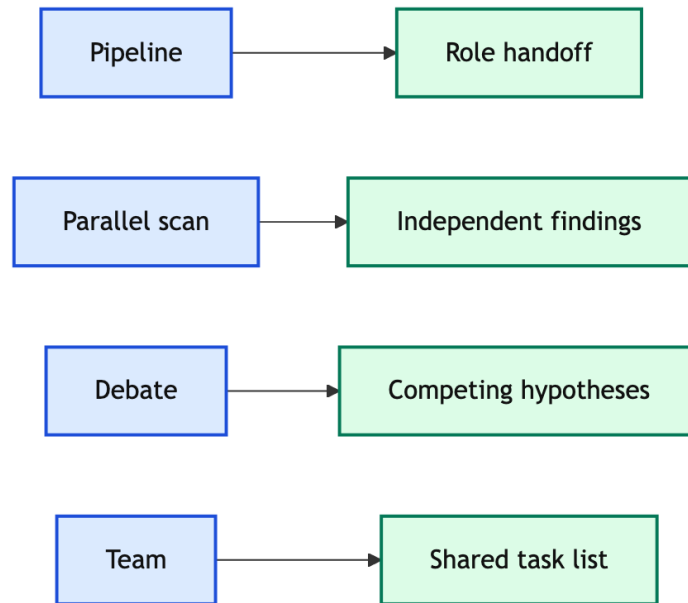


多 Agent 的第一性问题是可分解性。

如果任务不能被拆成独立判断或独立修改，多 agent 往往不会提升质量。它会让多个 agent 在同一块状态上互相踩踏，然后把“怎么写代码”的问题变成“怎么调度和合并多个不一致版本”的问题。

4 四种协作拓扑

不同论文和产品系统常用的协作方式，可以压缩成四种拓扑：pipeline、parallel scan、debate、team。



Pipeline 是角色流水线。ChatDev 把软件开发拆成 design、coding、testing；MetaGPT 用 SOP 把 Product Manager、Architect、Project Manager、Engineer、QA Engineer 串起来。这类系统强调 handoff：上游 agent 产生结构化中间件，下游 agent 按这个中间件继续。

Parallel scan 是并行检查。同一个 PR 可以让 security reviewer、performance reviewer、test reviewer 同时读；一个 bug 可以让多个 worker 分别调查日志、依赖、最近提交和运行环境。它的优点是减少等待并扩大证据面；缺点是最后必须有人去重、归并和判断优先级。

Debate 是竞争假设。根因不清时，可以要求 worker 明确提出不同 theory，并尝试反驳其他 theory。这比简单投票更有价值，因为 coding bug 常常不是“大家都觉得哪里像问题”，而是要找到能被复现和验证的因果链。

Team 是共享任务系统。Claude Code agent teams 明确有 lead、teammates、shared task list 和 mailbox。任务有 pending、in progress、completed 状态，还可以有依赖关系。这种拓扑最接近真实团队，但也最需要状态管理、权限、清理和监控。

拓扑	优势	主要风险
Pipeline	中间产物清楚，适合流程化任务	上游错误会级联
Parallel scan	证据面广，wall-clock 短	findings 重复或互相矛盾
Debate	减少单一路径早停	容易变成无证据辩论
Team	接近真实协作与任务队列	状态、权限和清理成本最高

5 MASAI：模块化不是聊天

MASAI 是 #12 最重要的研究例子，因为它展示了 multi-agent 不一定要靠自由对话。它把 repository-level issue repair 拆成五类 sub-agent：Test Template Generator、Issue Reproducer、Edit Localizer、Fixer、Ranker。每个 sub-agent 有明确目标、策略、动作空间和输出格式。它们不是在聊天室里互相说服，而是把上游输出作为下游输入。

MASAI 模块	主要工作	给下游的输出
Test Template Generator	学会怎样写和运行测试	test template / command
Issue Reproducer	构造能复现 issue 的测试	reproduction test
Edit Localizer	定位需要修改的代码区域	suspicious locations
Fixer	生成多个候选 patch	candidate patches
Ranker	用测试和模型判断排序	selected patch

这个设计的关键 insight 是：多 agent 可以用来减少 trajectory 膨胀。一个单体 agent 如果同时探索测试框架、复现 bug、定位代码、尝试 patch、运行验证，轨迹会变得很长，里面混入大量与当前决策无关的上下文。MASAI 把这些职责拆开，让不同 sub-agent 只处理自己的局部问题，再通过 typed output 串起来。

MASAI 论文报告在 SWE-bench Lite 上达到 28.33% resolution rate。这里不能把它当作当前 leaderboard 排名来读；更重要的是它的系统设计含义：多 agent 真正有用时，往往不是因为“大家一起聊天”，而是因为某些子问题需要不同策略、不同温度、不同动作空间、不同验证方式。

Insight. MASAI 给 coding agent 的启发是：模块化的核心是 objective 和 output contract，而不是角色名字。一个 sub-agent 如果不能交付可检查的中间产物，它就只是把主 agent 的不确定性挪到了别处。

6 ChatDev、MetaGPT 和 AutoGen

ChatDev 和 MetaGPT 更接近“模拟软件团队”的路线。ChatDev 用 chat chain 组织 design、coding、testing，并提出 communicative dehallucination，让 agents 在对话中要求更具体的实现、检查编译、补全缺失和减少 coding hallucination。MetaGPT 则把 SOP 写进 prompt sequence，让 Product Manager 产出 PRD，Architect 产出系统设计，Project Manager 分配任务，Engineer 写代码，QA Engineer 做测试和质量检查。

这两条路线共同说明：多 agent 的角色分工必须变成中间产物。Product Manager 不是一个名字，而是 PRD；Architect 不是一个人格，而是接口、文件结构和系统设计；QA 不是“挑毛病”，而是测试、失败用例和质量 gate。没有中间产物，角色分工只是 prompt 装饰。

AutoGen 提供的是更通用的抽象：conversable agents。一个 agent 可以由 LLM、人类、工具或它们的组合支撑；不同 agent 可以通过 conversation programming 定义控制流、终止条件、工具执行和 human input。对于 coding agent，这个抽象提醒我们：多 agent 系统不一定每个节点都是同一种模型。一个节点可能是 LLM reviewer，一个节点可能是 test runner，一个节点可能是人类批准，一个节点可能是静态分析工具。

系统	主要贡献	对 coding agent 的启发
ChatDev	chat chain + design/coding/testing	对话要有阶段和停止条件
MetaGPT	SOP + role deliverables	角色必须产出可用中间件
AutoGen	conversable agents + control flow	agent 可以混合 LLM、人和工具
MASAI	modular sub-agents for issue repair	子任务要有 objective 和 output

7 产品系统里的 Subagents 和 Teams

研究系统说明“多 agent 可以怎样组织”，产品系统说明“多 agent 怎样进入真实开发 workflow”。Codex subagents 文档强调手动触发：Codex 不会默认自动 spawn subagents，用户要明确说 spawn、delegate、

parallel。文档还提醒 subagent 会消耗更多 tokens，因为每个 subagent 都做自己的模型和工具工作。这和前面的协调成本模型一致。

Codex 侧的推荐用法很保守：先从 read-heavy tasks 开始，例如 exploration、tests、triage、summarization；对 write-heavy workflows 更谨慎，因为多个 agent 同时编辑代码会制造 conflicts 和 coordination overhead。这是产品系统给出的重要工程判断：并行阅读通常比并行写入更安全。

Claude Code agent teams 更像一个小型团队运行时。它有 lead、teammates、shared task list、mailbox。teammates 是独立 sessions，有自己的 context window，不继承 lead 的完整对话历史；它们通过 task list 和 messages 协调。文档还强调 plan approval、task dependencies、file locking、quality hooks、token usage、file conflicts、monitoring 和 experimental limitations。

这些细节说明：产品级 multi-agent 的核心不是“能开几个窗口”，而是是否有 team state。没有 task list，就不知道谁做什么；没有 mailbox，就不知道结果如何回传；没有 file ownership，就容易覆盖；没有 lead synthesis，就会得到一堆互相无关的 findings；没有 cleanup，就会留下孤儿 session 或状态残留。

8 什么时候使用多 Agent

一个实用判断是：如果子任务之间主要共享输入，不共享修改状态，多 agent 更可能有收益。比如并行 code review：三个 reviewer 看同一个 diff，但分别关注 security、performance、test coverage。它们共享输入 diff，却不需要同时修改同一个文件。最终 lead 汇总 findings，用户决定哪些要改。

另一个适合场景是 competing hypotheses。比如线上偶发超时，可能来自数据库索引、网络重试、锁竞争、缓存击穿或最近依赖升级。让一个 agent 只调查一种 theory，并要求它提供支持和反证，比让一个 agent 在一个长上下文里不断切换方向更稳。

第三个场景是 MASAI-style pipeline：复现、定位、修复、排序本来就是不同问题。一个 worker 负责读测试框架和写 reproduction test；另一个 worker 负责定位；另一个 worker 生成 patch；最后 ranker 用测试输出和风险判断选候选。这里的并行不一定完全同时发生，但模块化减少了上下文污染。

适合使用多 agent 的信号：

子任务可以独立完成；输出可以短摘要化；每个 worker 有不同 lens；不需要多人同时改同一文件；最终有明确 lead 或 human reviewer 负责合成。

8.1 例子：并行 PR Review

一个最适合作为团队起步实验的场景，是让多个 agent 并行 review 一个 PR。它比并行实现更安全，因为 worker 主要读代码、读测试、读设计文档，不直接改文件。一个 security reviewer 检查鉴权、输入校验、secret 和权限边界；一个 test reviewer 检查新增逻辑是否有覆盖、旧测试是否仍有效；一个 maintainability reviewer 检查复杂度、命名、重复代码和长期维护成本。

Reviewer	只负责什么	必须返回什么
Security	权限、输入、secret、注入风险	文件引用、风险等级、复现路径
Tests	缺失测试、脆弱测试、未覆盖路径	具体测试建议和现有覆盖证据
Maintainability	复杂度、边界、重复、命名	可维护性问题和简化建议
Lead	去重、排序、冲突判断	blocking list 和 final summary

这个 workflow 的关键不是让三个 reviewer 产生更多文字，而是让每个 reviewer 有不可重叠的 lens。输出最好被限制成固定格式：finding、file reference、evidence、risk、suggested action。Lead 只接受

这种结构化结果，不接受泛泛的“代码看起来不错”。这样最终 synthesis 可以按风险排序，而不是按 agent 发言顺序排序。

并行 PR review 也能暴露多 agent 的边界。如果三个 reviewer 都指出同一个问题，那是强信号；如果它们结论冲突，lead 必须回到 evidence，而不是做多数投票。如果 security reviewer 建议大改，而 maintainability reviewer 建议保持简单，人类 reviewer 要决定产品风险优先级。多 agent 提供证据，不替代 owner judgment。

9 什么时候不要使用

小任务不适合多 agent。修一个拼写错误、改一个配置字段、补一个明显单测，单 agent 加 review 够了。额外 spawn worker 只会增加启动、上下文加载和汇总成本。

强耦合写入不适合多 agent。比如重构核心鉴权模块、修改共享类型定义、迁移数据库 schema、调整跨服务协议。这些任务的关键风险在于一致性和全局判断，让多个 agent 同时写容易制造冲突。可以让多个 agent 做调查或 review，但 implementation 需要一个清晰 owner。

高风险操作也不适合自动多 agent。删除数据、推送分支、修改权限、联网安装依赖、处理 secret、改生产配置，都应该有严格 approval 和 sandbox。多个 agent 同时请求权限会产生 prompt fatigue，用户可能因为疲劳而误批。

任务类型	推荐形态	原因
PR review	多 reviewer + lead synthesis	lens 独立，输出可合成
根因调查	competing hypotheses	避免单一路径早停
大范围搜索	parallel scan	读多写少，冲突低
局部修复	single agent + review	协调成本不划算
核心重构	one owner + advisors	需要一致性判断
危险操作	human-controlled workflow	权限和责任边界更重要

10 Handoff 和 Synthesis

多 agent 系统最容易失败在 handoff。worker 返回“我看了一下，没问题”没有价值；worker 应该返回结构化结果：检查范围、证据、文件引用、结论、置信度、未检查内容、建议下一步。这样 lead 才能合成。

字段	含义
Scope	我检查了哪些文件、日志、测试、路径
Evidence	原始证据是什么，能否复现或链接
Finding	结论是什么，是否阻塞任务
Confidence	高/中/低，为什么
Open gaps	还有哪些没有检查
Action	建议修改、继续调查、忽略或交给人类

Lead 的职责不是把所有 worker 输出拼接起来，而是去重、归并、冲突解决、风险排序和生成可 review 的最终结果。一个好的 synthesis 会明确说明：哪些 findings 一致，哪些互相矛盾，哪些有证据，哪些只是猜测，哪些需要新的实验。

Reviewer/implementer split 是最有用的 multi-agent pattern 之一。Implementer 负责最小 patch；reviewer 负责从测试覆盖、安全、性能、可维护性角度检查；lead 或 human 负责决定哪些意见采纳。它比“两个 agent 都写 patch”更稳，因为它减少 merge conflict，同时增加独立审查。

10.1 任务切分粒度

任务切得太小，coordination overhead 会超过收益。比如把“给函数加一个边界条件”拆成 search、edit、test、review 四个 agents，启动和合成成本明显不划算。任务切得太大，worker 会长时间在自己的上下文里漂移，lead 很晚才发现方向错了。好的粒度通常满足三个条件：worker 可以在 10-30 分钟内产出可读结果；结果能用一个固定模板返回；失败时可以丢弃而不影响其他 worker。

粒度	现象	调整方式
太小	启动、权限、汇总比工作本身更重	合并为 single-agent 子任务
太大	worker 长时间无反馈，方向错误难发现	拆成 research / patch / review
正好	产物短、证据清楚、可独立验收	保持固定 output contract

Claude Code agent teams 文档建议从 research、review、library investigation 等边界清楚的任务开始，这个建议可以推广到所有 multi-agent coding：先让 agents 并行读和判断，再逐步开放并行写。因为读任务的失败通常表现为遗漏证据；写任务的失败会变成冲突 diff、破坏测试和权限风险。

任务粒度还决定模型选择和工具权限。快速扫描可以用低成本、低推理 worker；安全 review、根因分析、架构判断则需要更强 reasoning。工具也不应该默认一致：reviewer 可能只需要 read/grep/test，implementer 才需要 edit，release worker 才可能需要 git 或 PR 相关动作。多 agent 的好处之一，是可以让不同 worker 拥有不同能力边界。

Worker output contract:

一句结论；三条以内 evidence；文件或命令引用；风险等级；建议 action；明确说明未检查范围。

11 Worker 输出契约示例

多 agent 系统要可靠，worker 输出必须像 review note，而不是像聊天记录。下面是一个可复用的最小格式：

结论：发现一个可能阻塞的失败路径。

证据：运行 `pytest tests/auth/test_token.py::test_expired` 失败；错误来自 `auth/session.py:84`；当前 patch 未覆盖 refresh token。

风险：高，影响登录续期。

未检查：未跑 OAuth integration tests。

建议：先补 refresh token regression test，再改 session expiry branch。

这个格式看起来朴素，但它解决了三个问题。第一，lead 不需要读完整 transcript 就能判断 worker 贡献。第二，证据和未检查范围分开，避免 worker 把低置信度猜测写成事实。第三，action 是可执行的，lead 可以把它变成下一轮任务或 human review item。

输出字段	合格写法	不合格写法
结论	一个明确 finding 或 no finding	“看起来还行”
证据	命令、文件、日志、行号	“我觉得是这里”
风险	blocking / non-blocking / unknown	不区分严重程度
未检查	明确列出边界	默认自己全覆盖
建议	下一步动作	泛泛说继续调查

如果 worker 输出不能压缩成这种形式，就说明任务切分或工具授权不对。可能 worker 任务太大，也可能它缺少测试权限、文件访问权限或明确 objective。此时继续增加 worker 数量，只会增加不可合成的噪声。

12 Merge、权限和 Review

并行写代码时，workspace 边界决定成败。最简单的规则是 file ownership：每个 worker 拥有不同文件或模块。更稳的规则是 branch 或 worktree ownership：每个 worker 在独立 worktree 中产出 diff，由 lead 或人类逐个 review 和 cherry-pick。不要让多个 agent 在同一个 checkout 里同时编辑同一批文件。

权限也会随 agent 数量放大。Claude Code agent teams 文档说明 teammates start with lead 的 permission settings，并提醒 permission prompts 会 bubble up 到 lead。这个设计很合理，但也说明风险：如果 lead 运行在过宽权限下，worker 会继承风险；如果权限太细，多个 worker 会制造大量提示，导致 prompt fatigue。

Review 负担是另一个常被低估的成本。三个 worker 不是把 review 时间除以三，而是可能生成三倍 findings。系统必须帮用户压缩证据：按文件、风险、blocking/non-blocking、是否有测试、是否需要人类判断来组织，而不是把 worker transcript 全部丢给用户。

Insight. 多 agent 不是为了替代 review，而是为了产生更好的 review 输入。没有合成、去重、排序和证据压缩，多 agent 会把用户从“审一个 diff”变成“审一堆轨迹”。

13 冲突治理

多 agent 的冲突不只有 git merge conflict。更常见的是语义冲突：两个 worker 都修改不同文件，但背后的设计假设互相矛盾；或者 reviewer 认为应该最小 patch，implementer 却做了大范围重构。系统需要在 diff 合并之前识别这些冲突。

冲突类型	表现	处理方式
文件冲突 接口冲突	同一文件同一 hunk 被改 一个 worker 改签名，另一个仍按旧签名写测试	独立 worktree，逐个 cherry-pick lead 先冻结接口，再分发任务
假设冲突 范围冲突 权限冲突	根因解释不同 一个最小修，一个大重构 多 worker 请求高风险命令	要求各自给 reproduction evidence 人类 owner 决定 scope 串行审批，危险动作集中到 lead

一个实用规则是：多 agent 可以并行探索，但高影响写入应该串行化。先让 worker 独立收集证据和候选 patch，再由 lead 选择一个主线 diff，其他 worker 转为 review 和验证角色。这样既保留并行探索的好处，又避免多个 patch 同时改变系统假设。

14 Lead Agent 审计表

Lead agent 是多 agent 系统的责任中心。它不一定最强，也不一定亲自写最多代码，但它必须维护任务状态、worker 分工、证据合成、冲突处理和最终交付。没有 lead audit，多 agent 会退化成多个独立聊天窗口。

审计项	Lead 应该记录	失败信号
Task board	每个 worker 的目标、状态、owner	不知道谁在做什么
Evidence index	findings 按文件/风险聚合	重复 findings 淹没重点
Conflict log	哪些结论互相矛盾	冲突被静默覆盖
Decision record	采纳/拒绝哪些建议及原因	最终 patch 无来源
Review package	diff、测试、风险、未检查范围	用户必须读完整轨迹

这个审计表也能约束 worker 数量。若 lead 无法维护 task board 和 evidence index，说明 worker 已经太多，或者任务切分不清楚。多 agent 的瓶颈往往不是生成能力，而是 lead synthesis 的带宽。系统设计时要保护这个带宽：限制 worker 数量，压缩输出格式，提前定义冲突处理规则，避免让 lead 消化原始 transcript。

14.1 成本报告样例

多 agent 的结果报告应该显式说明“多出来的 agent 买到了什么”。一个合格报告不能只写 resolved rate 上升，还要写总 token、wall time、工具调用、review 时间和冲突率。否则系统可能只是用更多预算换了更多尝试。

字段	示例值	解释
Workers	lead + 2 reviewers	不是无限并发
Wall time	18 min vs 31 min	是否减少等待
Total tokens	2.4x baseline	成本是否可接受
Findings	3 blocking, 5 non-blocking	review 输入是否更好
Conflicts	0 merge, 1 hypothesis conflict	协调副作用
Human time	9 min review	用户是否真的省力

如果一个 multi-agent system 不能给出这种报告，就很难判断它是否值得。尤其在工程团队里，节省 wall time 但增加大量 review 时间，不一定是净收益；提高 resolved rate 但产生更多高风险权限请求，也可能不适合上线。

15 怎样评估多 Agent 是否真的有用

评价 multi-agent coding，不能只看最终任务是否完成。因为多 agent 通常同时增加了模型调用次数、总 token、工具运行次数、探索路径和人类 review 量。如果一个三 agent 系统比单 agent 高 5 个百分点，但用了三倍预算、产生两倍 review 时间和更多冲突，它未必是更好的系统。

更公平的报告方式，是把质量、成本和协作副作用一起记录。

维度	应该报告什么	为什么重要
质量	resolved / tests / reviewer findings	是否真的改善结果
预算	token、模型调用、工具调用、wall time	避免把预算当能力
冲突	文件重叠、merge conflict、rework	衡量并行写入成本
证据	日志、diff、测试、未检查范围	判断是否可 review
人力	review 时间、批准次数、介入次数	衡量对团队真实负担

这里有一个重要区分：**more attempts** 和 **better collaboration** 不是一回事。如果一个系统只是让五个 agent 分别尝试同一个 patch，然后选一个通过测试的结果，它可能有效，但它更像 parallel

sampling。如果 worker 有不同任务、不同证据、不同输出契约，并且 lead 能合成互补结果，这才是 collaboration。两者都可以有价值，但不能混在一起解释。

最干净的实验设计，是固定总预算后比较 single-agent、parallel sampling、role-based pipeline 和 team workflow。这样才能判断提升来自更多尝试，还是来自更好的 decomposition。对于产品团队，最实用的指标则是：是否减少人类等待时间，是否提高 review 输入质量，是否降低遗漏风险，是否没有显著增加冲突和误批。

16 局限与开放问题

第一，token 和成本会快速增长。每个 agent 都需要自己的上下文、推理和工具调用。并行可以降低 wall-clock time，但不一定降低总成本。

第二，false consensus 很危险。多个 agent 都同意一个错误结论，不代表结论正确，可能只是共享了同一个错误 prompt 或同一个缺失证据。多 agent 需要独立 lens 和反证要求，否则只是重复同一种偏见。

第三，team state 会变复杂。Claude Code agent teams 文档列出的 limitations 很有代表性：session resumption、task status lag、shutdown、one team at a time、no nested teams、fixed lead、permission inheritance、display mode 等，都是产品级 team runtime 必须处理的工程细节。

第四，parallel implementation 容易制造 ownership 问题。谁对最终 patch 负责？谁保证跨文件一致性？谁决定冲突怎么解？如果答案不清楚，多 agent 会削弱责任边界。

第五，multi-agent 评测本身不简单。成功率提升可能来自更大 token budget、更多工具调用、更多尝试次数、更多模型调用，而不一定来自协作结构本身。评价时要把并行数量、总预算、review 人力、冲突率、最终质量一起报告。

17 最终 Takeaway

多 agent coding 的正确问题不是“要不要多开几个 agent”，而是：这个任务是否能拆成独立工作单元？每个 worker 是否有清楚 objective？输出是否可验证？lead 是否能合成？diff 是否能隔离？review 是否能承受？

如果答案是 yes，多 agent 可以带来更快的探索、更广的证据、更干净的上下文和更强的 reviewer/implementer 分工。如果答案是 no，多 agent 只会增加成本、冲突和幻觉式一致。

设计自测：

每个 worker 的输入和输出是否一页内说清？
worker 是否能不改代码也贡献证据？
是否有明确的 owner 合成最终结果？
多个 diff 是否能独立 review、独立撤销、独立合并？

参考资料

- [1] [MASAI](#). modular sub-agents for software-engineering issue repair and SWE-bench Lite result
- [2] [ChatDev](#). chat chain, design/coding/testing phases, and communicative dehallucination
- [3] [MetaGPT](#). SOP-driven role specialization and software-company style collaboration
- [4] [AutoGen](#). conversable agents and conversation programming for multi-agent workflows
- [5] [Codex Subagents](#). official guidance on subagent workflows, read-heavy tasks, and coordination cost

- [6] **Claude Code Agent Teams**. lead/teammate architecture, shared task list, mailbox, and limitations
- [7] **Claude Code Subagents**. subagent definitions, independent work, tool/model specialization, and scopes